LAB 03_REPORT

OOP

OCTOBER 30, 2026

NAME: SIEM KHATTYAVATEY
Student ID: p20250056

Practice 1: Library Management

Problem: Design a program to manage a book in a library

1. Flowchart

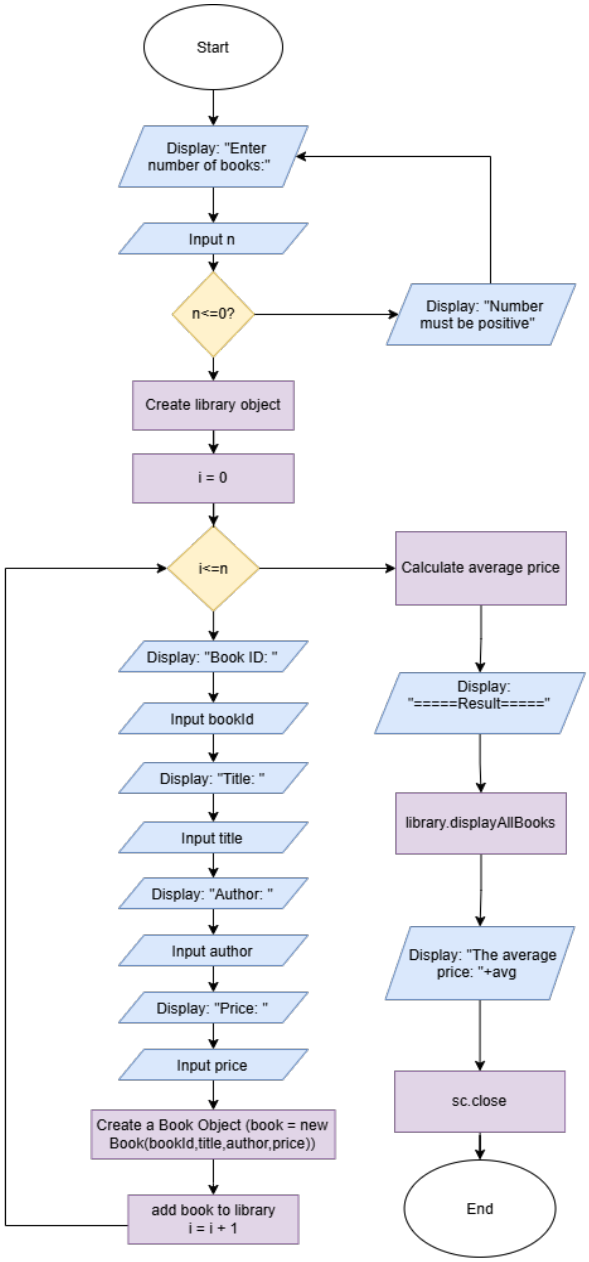


Figure 1: Flowchart

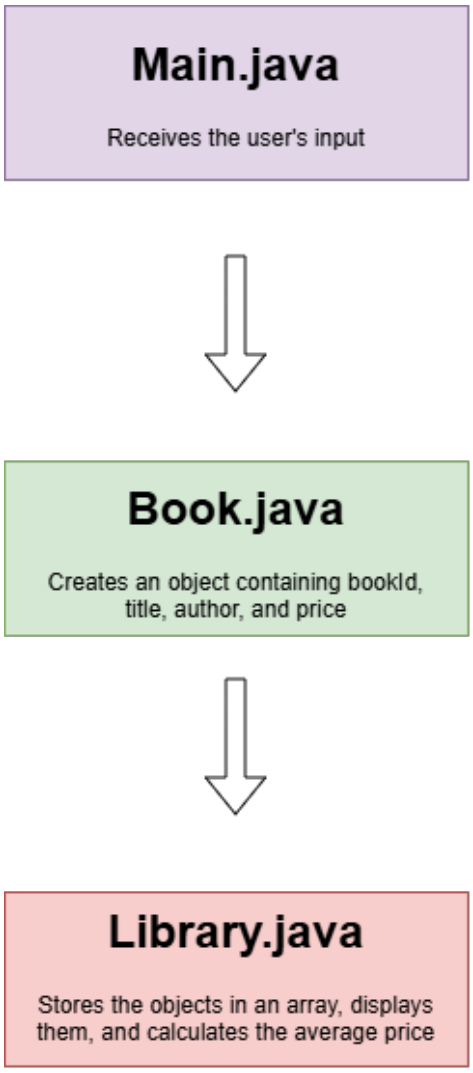


Figure 2: Structure of how classes work together

2. Pseudocode: Library Management System

```
START

    DISPLAY "Enter number of books: "
    INPUT n

    IF n <= 0 THEN
        DISPLAY "Number of books must be positive."
        END
    END IF

    CREATE Library with size n

    FOR i = 0 TO n - 1 DO

        DISPLAY "Enter details for book ", i + 1

        DISPLAY "Book ID: "
        INPUT id

        DISPLAY "Title: "
        INPUT title

        DISPLAY "Author: "
        INPUT author

        DISPLAY "Price: "
        INPUT price

        CREATE Book(id, title, author, price)

        ADD Book to Library

    END FOR

    DISPLAY "===== LIBRARY BOOKS ====="

    CALL library.displayAllBooks()

    averagePrice = library.calculateAveragePrice()

    DISPLAY "Average price: $", averagePrice

    CLOSE input

END
```

3. Sample solution:

Main.java: Used to receive input from the user

```java
package Practice_1;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of books: ");
        int n = sc.nextInt();
        sc.nextLine();

        if (n <= 0) {
            System.out.println("Number of books must be positive.");
            sc.close();
            return;
        }

        Library library = new Library(n);

        // Input book information
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for book " + (i + 1));

            System.out.print("Book ID: ");
            String id = sc.nextLine();

            System.out.print("Title: ");
            String title = sc.nextLine();

            System.out.print("Author: ");
            String author = sc.nextLine();

            System.out.print("Price: ");
            double price = sc.nextDouble();
            sc.nextLine();

            Book book = new Book(id, title, author, price);
            library.addBook(book);
        }

        // Display results
        System.out.println("\n===== LIBRARY BOOKS =====");
        library.displayAllBooks();

        System.out.printf(
                "%nAverage price: $%.2f%n",
                library.calculateAveragePrice()
        );
        sc.close();
    }
}
```

- **Library library = new Library(n):** creates a Library object with a capacity to store the specified number of books.
- **Book book = new Book(id, title, author, price):** creates a Book object containing the book's ID, title, author, and price.
- **library.calculateAveragePrice():** calculates the average price of all books

Book.java: Creates objects that contain bookId, title, author, and price

```java
package Practice_1;

public class Book {  5 usages
    private String bookID;  3 usages
    private String title;  3 usages
    private String author;  3 usages
    private double price;  3 usages

    // Constructor
    public Book(String bookID, String title, String author, double price) {  1 usage
        this.bookID = bookID;
        this.title = title;
        this.author = author;
        this.price = price;
    }

    // Getters
    public String getBookID() {  no usages
        return bookID;
    }

    public String getTitle() {  no usages
        return title;
    }

    public String getAuthor() {  no usages
        return author;
    }

    public double getPrice() {  1 usage
        return price;
    }

    // Display book information
    public void displayBook() {  1 usage
        System.out.println("Book ID: " + bookID);
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.printf("Price: $%.2f%n", price);
    }
}
```

- **public Book(String bookID, String title, String author, double price):** initializes a new Book object with the information provided.
- **this: Statements such as this.title = title:** assigns the parameter value to the object's
- **displayBook():** displays the book's ID, title, author, and price.
- The attributes are declared private, so they cannot be accessed directly from outside the Book class.

Library.java: Stores the objects in an array and calculates the average price.

```java
package Practice_1;

public class Library {  2 usages
    private Book[] books;  5 usages
    private int count;  8 usages

    // Constructor
    public Library(int size) {  1 usage
        books = new Book[size];
        count = 0;
    }

    // Add a book
    public void addBook(Book book) {  1 usage
        if (count < books.length) {
            books[count] = book;
            count++;
        }else {
            System.out.println("Book list is full!");
        }
    }

    // Display all books
    public void displayAllBooks() {  1 usage
        for (int i = 0; i < count; i++) {
            System.out.println("\nBook " + (i + 1));
            books[i].displayBook();
        }
    }

    // Calculate average price
    public double calculateAveragePrice() {  1 usage
        if (count == 0) {
            return 0;
        }
        double total = 0;
        for (int i = 0; i < count; i++) {
            total += books[i].getPrice();
        }
        return total / count;
    }
}
```

- **public void displayAllBooks():** uses a for loop to go through the stored books and display each book's information by calling **books[i].displayBook()**.
- **total += books[i].getPrice():** retrieves each book's price using getPrice() and adds it to the total.

Output:

```
Enter number of books: 2

Enter details for book 1
Book ID: j25372
Title: King game
Author: R.J
Price: 59

Enter details for book 2
Book ID: k43297
Title: Fade away
Author: Rebecca Smith
Price: 12
```

```
===== LIBRARY BOOKS =====

Book 1
Book ID: j25372
Title: King game
Author: R.J
Price: $59.00

Book 2
Book ID: k43297
Title: Fade away
Author: Rebecca Smith
Price: $12.00

Average price: $35.50
```

```
Enter number of books: 2

Enter details for book 1
Book ID: L7430238
Title: Think
Author: UNKNOWN
Price: 44

Enter details for book 2
Book ID: M3482939
Title: Big
Author: J.Kevin
Price: 32
```

```
===== LIBRARY BOOKS =====

Book 1
Book ID: L7430238
Title: Think
Author: UNKNOWN
Price: $44.00

Book 2
Book ID: M3482939
Title: Big
Author: J.Kevin
Price: $32.00

Average price: $38.00
```

Practice 2: Temperature Conversion

Problem: Write a program that reads the temperature of N cities in Celsius and converts them to Fahrenheit

1. **Flowchart**

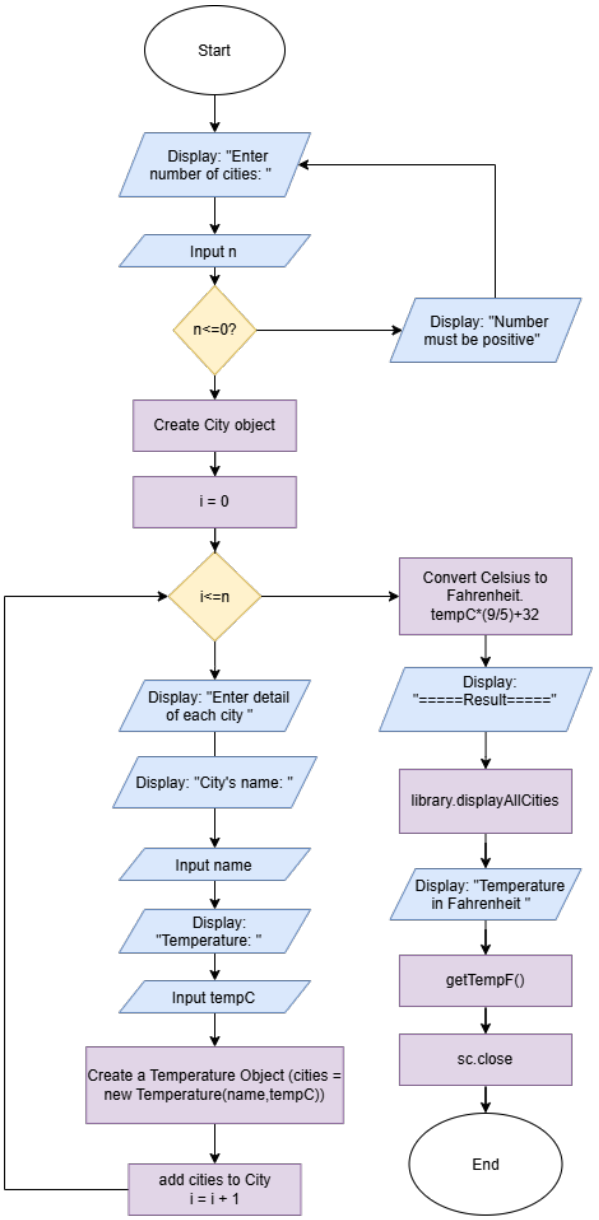


Figure 3: Flowchart

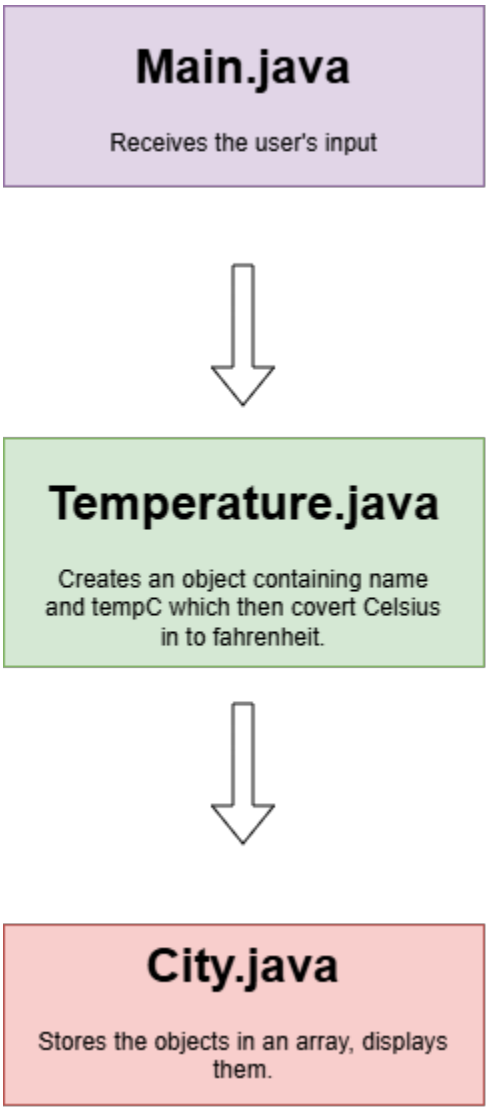


Figure 4: Structure of how classes work together

2. Pseudocode:

```
START

    DISPLAY "Enter number of cities: "
    INPUT n

    IF n <= 0 THEN
        DISPLAY "Number of books must be positive."
        END
    END IF

    CREATE City with size n

    FOR i = 0 TO n  DO

        DISPLAY "Enter details for city ", i + 1

        DISPLAY "City's name: "
        INPUT name

        DISPLAY "Temperature: "
        INPUT tempC

        CREATE Temperature(name,tempC)

        ADD Temperature to City

    END FOR

    DISPLAY "======= RESULTS ======="

    CALL library.displayAllCities()

    getTempF(){
        return(TempC*(9/5) + 32 )
    }

    DISPLAY "Temperature in Fahrenheit: F", getTempF()

    CLOSE input

END
```

3. Sample solution:

Main.java: Receive input from User

```java
package Practice_2;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of cities: ");
        int n = sc.nextInt();
        sc.nextLine();

        if (n <= 0) {
            System.out.println("Number must be positive!");
            sc.close();
            return;
        }

        City City = new City(n);

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for city " + (i + 1));

            System.out.print("City name: ");
            String name = sc.nextLine();

            System.out.print("Temperature in Celsius: ");
            double tempC = sc.nextDouble();
            sc.nextLine();

            Temperature city = new Temperature(name, tempC);
            City.addCity(city);
        }

        System.out.println("\n===== RESULTS =====");
        City.displayAllCities();

        sc.close();
    }
}
```

- **City City = new City(n):** creates a City object with capacity for n cities.
- The for loop repeats the input process for each city, asking for its name and temperature in Celsius.
- **Temperature city = new Temperature(name, tempC):** creates a Temperature object containing the city name and its Celsius temperature.
- **City.addCity(city):** adds that object to the City collection.

Temperature.java: Creates objects that contain name and tempC.

Then convert Celsius to Fahrenheit.

```java
package Practice_2;

    public class Temperature {  5 usages

        private String name;  3 usages
        private double tempC;  4 usages

        // Constructor
        public Temperature(String name, double tempC) {  1 usage
            this.name = name;
            this.tempC = tempC;
        }

        // Getters
        public String getName() {  no usages
            return name;
        }

        public double getTempc() {  no usages
            return tempC;
        }

        //Convert Celsius to Fahrenheit
        public double getTempF() {  1 usage
            return (tempC * 9.0 / 5.0) + 32;
        }

        // Display one city information
        public void displayCities() {  1 usage
            System.out.println("City's name: " + name);
            System.out.printf("Temperature: %.2f °C%n", tempC);
            System.out.printf("Fahrenheit: %.2f °F%n", getTempF());;
            System.out.println("-------------------");
        }
    }
```

- **Temperature(String name, double tempC):** Initializes a new Temperature object.
- **getName()** and **getTempC ():** Allow other classes to retrieve the private variables' values.
- **getTempF():** Converts Celsius into Fahrenheit using the formula:

  °F = (°C \times 9/5) + 32

- **displayCities():** Displays the city's name, Celsius temperature, and Fahrenheit temperature.

City.java: Store objects in the array and display them.

```java
package Practice_2;

public class City {  2 usages
    private Temperature[] cities;  4 usages
    private int count;  5 usages

    // Constructor
    public City(int size) {  1 usage
        cities = new Temperature[size];
        count = 0;
    }

    // Add a city
    public void addCity(Temperature city) {  1 usage
        if (count < cities.length) {
            cities[count] = city;
            count++;
        } else {
            System.out.println("City list is full!");
        }
    }

    // Display all cities
    public void displayAllCities() {  1 usage
        for (int i = 0; i < count; i++) {
            System.out.println("\nCity " + (i + 1));
            cities[i].displayCities();
        }
    }
}
```

- **City(int size):** Initializes a new City object with a specified storage capacity.
- **cities = new Temperature[size]:** creates an array that can hold the specified number of Temperature objects.
- **addCity(Temperature city):** Adds a Temperature object to the array.
- **cities[i].displayCities(): calls the displayCities()** method from the Temperature class to display the city's name and temperatures.

Output:

```
Enter the number of cities: 4

Enter details for city 1
City name: Phnom Penh
Temperature in Celsius: 27

Enter details for city 2
City name: Takeo
Temperature in Celsius: 32

Enter details for city 3
City name: Battambong
Temperature in Celsius: 28

Enter details for city 4
City name: Kondal
Temperature in Celsius: 29

===== RESULTS =====

City 1
City's name: Phnom Penh
Temperature: 27.00 °C
Fahrenheit: 80.60 °F
-------------------

City 2
City's name: Takeo
Temperature: 32.00 °C
Fahrenheit: 89.60 °F
-------------------

City 3
City's name: Battambong
Temperature: 28.00 °C
Fahrenheit: 82.40 °F
-------------------

City 4
City's name: Kondal
Temperature: 29.00 °C
Fahrenheit: 84.20 °F
-------------------

Process finished with exit code 0
```

Practice 3: Employee Salary Management

Problem: Store and display the salaries of N employees

1. **Flowchart**

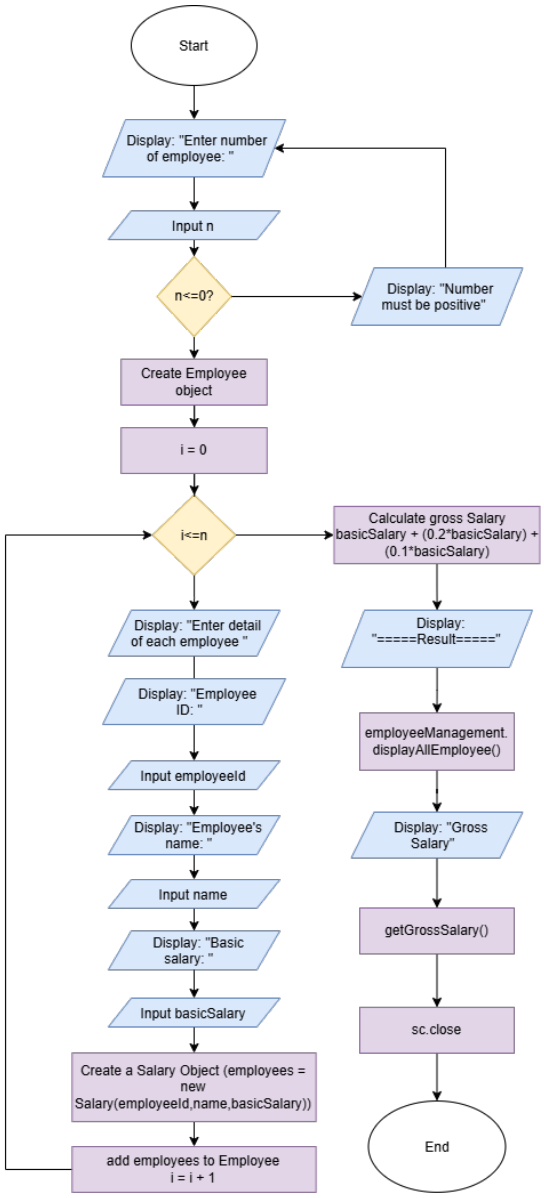


Figure 5: Flowchart

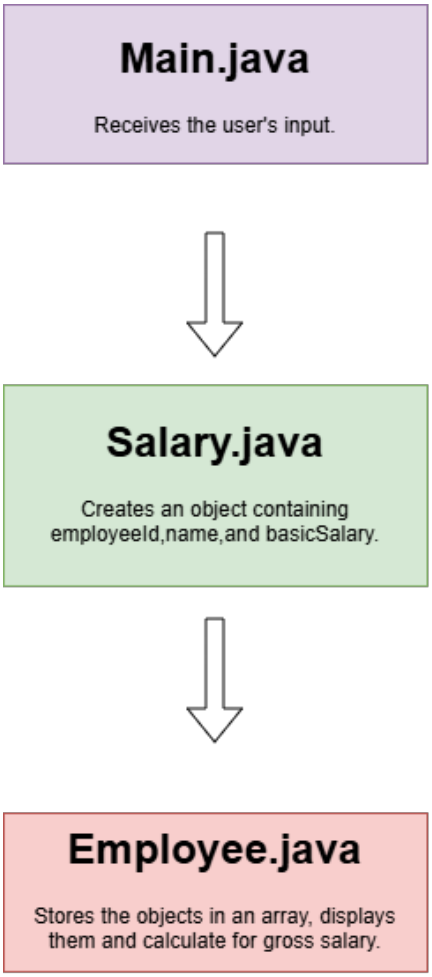


Figure 6: Structure of how classes work together

2. Pseudocode:

Practice_3.md

```
START

    DISPLAY "Enter number of employee: "
    INPUT n

    IF n <= 0 THEN
        DISPLAY "Number of must be positive."
        END
    END IF

    CREATE City with size n

    FOR i = 0 TO n  DO

        DISPLAY "Enter details for each employee ", i + 1

        DISPLAY "Employee ID: "
        INPUT employeeId

        DISPLAY "Employee's name: "
        INPUT name

        DISPLAY "Basic salary: "
        INPUT basicSalary

        CREATE Salary(employeeId,name,basicSalary)

        ADD Salary to Employee

    END FOR

    DISPLAY "======= RESULTS ======="

    CALL employeemanagement.displayAllEmployee()

    getGrossSalary(){
        return(basicSalary +(0.2 * basicSalary) + (0.1 * basicSalary))
    }

    DISPLAY "Gross Salary: $", getGrossSalary()

    CLOSE input

END
```

3. Sample solution:

Main.java: The program asks for the number of employees, validates the number, collects each employee's ID, name, and basic salary, stores the records using the Employee class, and displays the results.

```java
package Practice_3;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of employee: ");
        int n = sc.nextInt();
        sc.nextLine();

        if (n <= 0) {
            System.out.println("Number must be positive!");
            sc.close();
            return;
        }

        Employee employeeManagement = new Employee(n);

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details of employee " + (i + 1));

            System.out.print("Employee ID: ");
            String employeeId = sc.nextLine();

            System.out.print("Employee name: ");
            String name = sc.nextLine();

            System.out.print("Basic Salary: ");
            double basicSalary = sc.nextDouble();
            sc.nextLine();

            Salary employee = new Salary(employeeId, name, basicSalary);
            employeeManagement.addEmployee(employee);
        }

        System.out.println("\n===== RESULTS =====");
        employeeManagement.displayAllEmployee();

        sc.close();
    }
}
```

- **Employee employeeManagement = new Employee(n):** creates an Employee object with a capacity of n employees.
- **Salary employee = new Salary(employeeId, name, basicSalary):** creates a Salary object containing the employee's ID, name, and basic salary.

Salary.java: The Salary class stores employee information, provides getter methods to access the data, calculates gross salary by adding the 20% HRA and 10% TCA allowances, and displays the employee's salary details.

```java
package Practice_3;

public class Salary {  5 usages
    private String employeeId;  3 usages
    private String name;  3 usages
    private double basicSalary;  6 usages

    // Constructor
    public Salary(String employeeId, String name, double basicSalary) {  1 usage
        this.employeeId = employeeId;
        this.name = name;
        this.basicSalary = basicSalary;
    }

    // Getters
    public String getEmployeeId() {  no usages
        return employeeId;
    }

    public String getName() {  no usages
        return name;
    }

    public double getBasicSalary() {  no usages
        return basicSalary;
    }

    //Calculate for Gross Salary
    //Gross Salary = basic Salary + HRA(20% basic) + TCA(10% basic)
    public double getGrossSalary() {  1 usage
        return (basicSalary+ (0.2*basicSalary) + (0.1*basicSalary));
    }

    // Display one employee information
    public void displayEmployee() {  1 usage
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Employee's name: " + name);
        System.out.printf("Basic Salary: $%.2f%n", basicSalary);
        System.out.printf("Gross Salary: $%.2f%n", getGrossSalary());;
        System.out.println("--------------------");
    }
}
```

- **getEmployeeId(), getName(), getBasicSalary():** These methods allow other classes to retrieve the private variables' values.
- **getGrossSalary():** Calculates the employee's gross salary by adding the basic salary, Housing Rent Allowance (HRA), and Transport Allowance (TCA).
- **displayEmployee():** Displays the employee's ID and name.

Employee.java: The Employee class stores and manages multiple employee records, checks whether the array has available space, and displays the salary information of every employee stored in the array.

```java
package Practice_3;

public class Employee {  2 usages
    private Salary[] employees;  4 usages
    private int count;  5 usages

    // Constructor
    public Employee(int size) {  1 usage
        employees = new Salary[size];
        count = 0;
    }

    // Add an employee
    public void addEmployee(Salary employee) {  1 usage
        if (count < employees.length) {
            employees[count] = employee;
            count++;
        } else {
            System.out.println("Employee list is full!");
        }
    }

    // Display all Employee
    public void displayAllEmployee() {  1 usage
        for (int i = 0; i < count; i++) {
            employees[i].displayEmployee();
        }
    }
}
```

- **Salary[] employees:** creates an array to store Salary objects containing employee information.
- **Employee(int size):** Initializes an Employee object.
- **employees = new Salary[size]:** creates an array that can hold the specified number of employee records
- **employees[i].displayEmployee(): calls the displayEmployee()** method from the Salary class to display each employee's ID, name, basic salary, and gross salary.

Output:

```
Enter the number of employee: 3

Enter details of employee 1
Employee ID: Khmer4768
Employee name: Sun Sarak
Basic Salary: 589

Enter details of employee 2
Employee ID: Khmer3297
Employee name: Chum Neang
Basic Salary: 629

Enter details of employee 3
Employee ID: Khmer3279
Employee name: Tai Dara
Basic Salary: 489
```

```
===== RESULTS =====
Employee ID: Khmer4768
Employee's name: Sun Sarak
Basic Salary: $589.00
Gross Salary: $765.70
-------------------
Employee ID: Khmer3297
Employee's name: Chum Neang
Basic Salary: $629.00
Gross Salary: $817.70
-------------------
Employee ID: Khmer3279
Employee's name: Tai Dara
Basic Salary: $489.00
Gross Salary: $635.70
-------------------
```

Example1

```
Enter the number of employee: 3

Enter details of employee 1
Employee ID: l3y324
Employee name: Piruth
Basic Salary: 3482

Enter details of employee 2
Employee ID: tl3udad
Employee name: Kahna
Basic Salary: 6934

Enter details of employee 3
Employee ID: pwcqnwx
Employee name: Yuth
Basic Salary: 2389
```

```
===== RESULTS =====
Employee ID: l3y324
Employee's name: Piruth
Basic Salary: $3482.00
Gross Salary: $4526.60
-------------------
Employee ID: tl3udad
Employee's name: Kahna
Basic Salary: $6934.00
Gross Salary: $9014.20
-------------------
Employee ID: pwcqnwx
Employee's name: Yuth
Basic Salary: $2389.00
Gross Salary: $3105.70
-------------------
```

Example2

Practice 4: Student Grade Calculation

Problem: Calculate and display students' grades based on marks

1. Flowchart

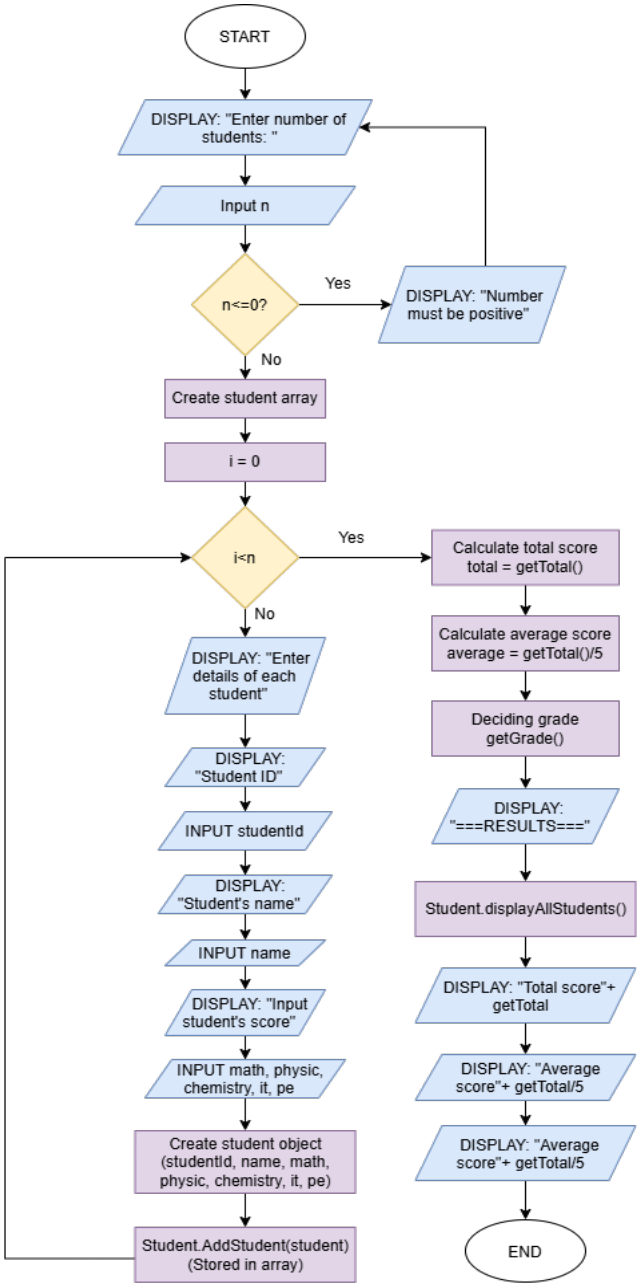


Figure 7: Flowchart

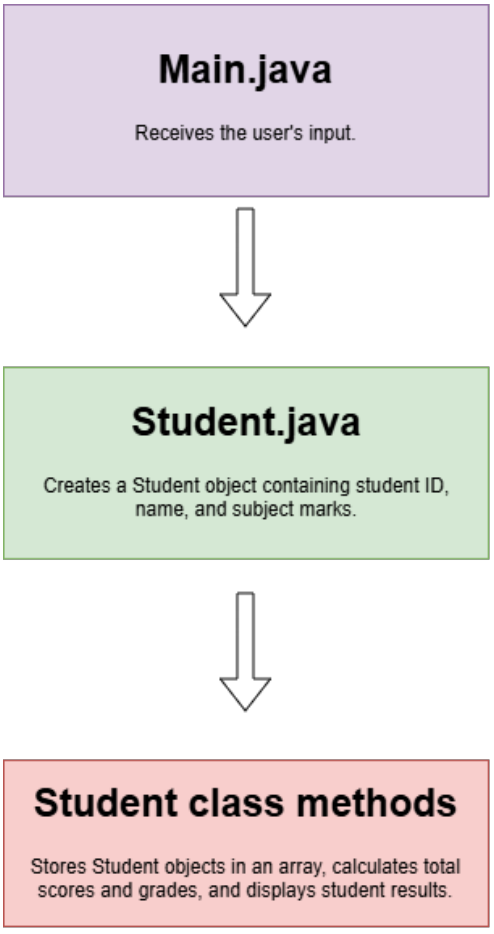


Figure 8: Structure of how classes work together

2. Pseudocode:

```
START

    INPUT numberOfStudents

    IF numberOfStudents <= 0 THEN
        DISPLAY "Number must be positive!"
        STOP
    END IF

    INITIALIZE Student array with numberOfStudents

    FOR each student from 1 to numberOfStudents DO

        INPUT studentId
        INPUT name

        INPUT math
        INPUT physic
        INPUT chemistry
        INPUT it
        INPUT pe

        CREATE Student object using the input data

        ADD Student object to the array

    END FOR

    DISPLAY "===== RESULTS ====="

    FOR each student in the array DO

        CALCULATE total = math + physic + chemistry + it + pe

        CALCULATE average = total / 5

        IF average >= 90 THEN
            grade = "A"
        ELSE IF average >= 75 THEN
            grade = "B"
        ELSE IF average >= 50 THEN
            grade = "C"
        ELSE
            grade = "F"
        END IF

        DISPLAY studentId
        DISPLAY name
        DISPLAY total
        DISPLAY average
        DISPLAY grade

    END FOR

END
```

3. Sample solution

Main.java: Receive input from User

```java
package Practice_4;

import java.util.Scanner;

public class Main {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter the number of Student: ");
            int n = sc.nextInt();
            sc.nextLine();

            if (n <= 0) {
                System.out.println("Number must be positive!");
                sc.close();
                return;
            }

            // Initialize the student array
            Student.initialize(n);

            // Input Student marks of each subject
            for (int i = 0; i < n; i++) {
                System.out.println("\nEnter mark for each student " + (i + 1));

                System.out.print("Student ID: ");
                String studentId = sc.nextLine();

                System.out.print("Student's name: ");
                String name = sc.nextLine();

                System.out.println("Input student's score");

                System.out.print("Math: ");
                double math = sc.nextDouble();
                sc.nextLine();

                System.out.print("Physic: ");
                double physic = sc.nextDouble();

                System.out.print("Chemistry: ");
                double chemistry = sc.nextDouble();

                System.out.print("IT: ");
                double it = sc.nextDouble();

                System.out.print("PE: ");
                double pe = sc.nextDouble();
                sc.nextLine();
                // Create and add a student
                Student student = new Student(studentId, name, math, physic, chemistry, it, pe);
                Student.addStudent(student);
            }

            // Display results
            System.out.println("\n===== RESULTS =====");
            Student.displayAllStudents();
            sc.close();
        }
    }
```

- **Scanner sc = new Scanner(System.in):** creates a Scanner object to receive input from the keyboard.
- **int n = sc.nextInt()**: reads and stores the number of students entered by the user.
- **for (int i = 0; i < n; i++)**: repeats the input process for each student until all students have been entered.
- **Student student = new Student(studentId, name, math, physic, chemistry, it, pe):** creates a student object containing the student's ID, name, and subject marks.

Student.java: Creates a Student object containing student ID, name, and subject marks.

Stores Student objects in an array, calculates total scores and grades, and displays student results

```java
package Practice_4;

public class Student {  8 usages

    private String studentId;  2 usages
    private String name;  2 usages
    private double math;  2 usages
    private double physic;  2 usages
    private double chemistry;  2 usages
    private double it;  2 usages
    private double pe;  2 usages

    private static Student[] students;  4 usages
    private static int count = 0;  5 usages

        // Constructor for each student;
        public Student(String studentId,String name, double math,  1 usage
                       double physic,double chemistry, double it, double pe) {
            this.studentId = studentId;
            this.name = name;
            this.math = math;
            this.physic = physic;
            this.chemistry = chemistry;
            this.it = it;
            this.pe = pe;
        }

        // Initialize the array
        public static void initialize(int size) {  1 usage
            students = new Student[size];
            count = 0;
        }

        // Add a student
        public static void addStudent(Student student) {  1 usage
            if (count < students.length) {
                students[count] = student;
                count++;
            } else {
                System.out.println("The number of student is full!");
            }
        }
        // Calculate for total score
        public double getTotal() { return (math + physic + chemistry + it + pe); }
        //Deciding grade
        public String getGrade() {  1 usage
            double average = getTotal() / 5;
```

```java
            if (average >= 90) {
                return "A";
            } else if (average >= 75) {
                return "B";
            } else if (average >= 50) {
                return "C";
            }else {
                return "F";
            }
        }
        // Display one student
        public void displayStudent() {  1 usage
            System.out.println("Student ID: " + studentId);
            System.out.println("Student's name: " + name);
            System.out.printf("Total: %.2f %n", getTotal());
            System.out.printf("Average: %.2f %n",getTotal()/5);
            System.out.printf("Grade: %s %n",getGrade());
        }

        // Display all students
        public static void displayAllStudents() {  1 usage
            for (int i = 0; i < count; i++) {
                System.out.println("\nStudent " + (i + 1));
                students[i].displayStudent();
            }
        }

}
```

- **public void displayStudent():** displays the student's ID, name, total marks, average marks, and grade.
- **public static void displayAllStudents():** uses a for loop to display the information of every student stored in the array.

Output:

```
Enter the number of Student: 2

Enter mark for each student 1
Student ID: p20250078
Student's name: John Wick
Input student's score
Math: 56
Physic: 98
Chemistry: 40
IT: 37
PE: 28

Enter mark for each student 2
Student ID: p2025
Student's name: Chann Dara
Input student's score
Math: 75
Physic: 79
Chemistry: 10
IT: 23
PE: 45
```

Input from user

```
===== RESULTS =====

Student 1
Student ID: p20250078
Student's name: John Wick
Total: 259.00
Average: 51.80
Grade: C

Student 2
Student ID: p2025
Student's name: Chann Dara
Total: 232.00
Average: 46.40
Grade: F

Process finished with exit code 0
```

Display the result

Practice 5: Simple banking system

Problem: Simulate a banking system.

1. Flowchart

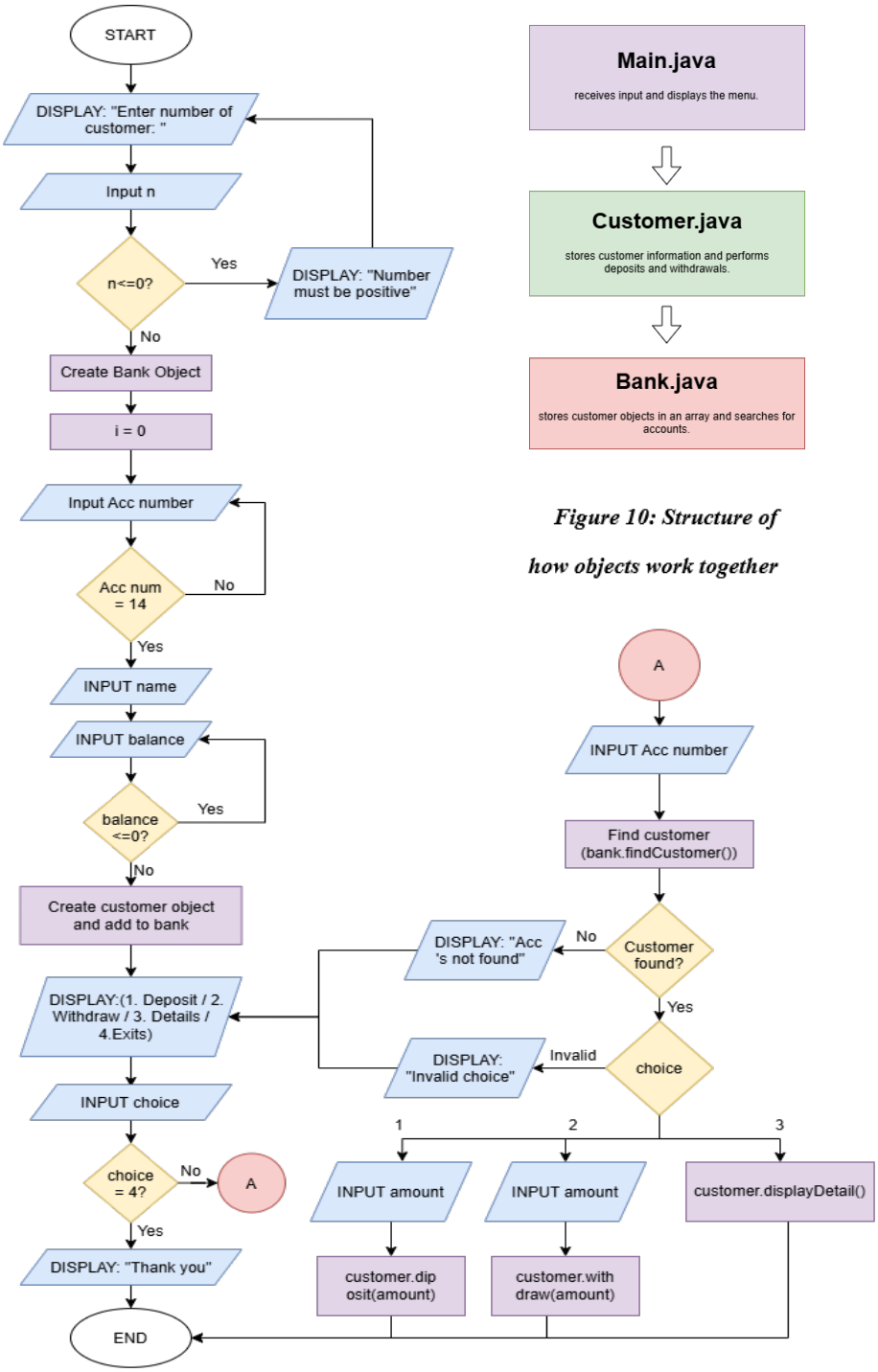


Figure 9: Flowchart

Figure 10: Structure of how objects work together

2. Pseudocode:

```
START

    INPUT numberOfCustomers

    IF numberOfCustomers <= 0 THEN
        DISPLAY "Number must be positive"
        STOP
    END IF

    CREATE Bank with numberOfCustomers

    FOR each customer from 1 to numberOfCustomers DO

        INPUT accountNumber
        WHILE accountNumber is not 14 digits DO
            DISPLAY "Invalid account number"
            INPUT accountNumber
        END WHILE

        INPUT name
        INPUT balance

        WHILE balance < 0 DO
            DISPLAY "Balance cannot be negative"
            INPUT balance
        END WHILE

        CREATE Customer object
        ADD Customer object to Bank

    END FOR

    REPEAT
        DISPLAY "===== BANKING SYSTEM ====="
        DISPLAY "1. Deposit"
        DISPLAY "2. Withdraw"
        DISPLAY "3. Display Account Details"
        DISPLAY "4. Exit"

        INPUT choice

        IF choice >= 1 AND choice <= 3 THEN

            INPUT accountNumber
            FIND customer using accountNumber

            IF customer is not found THEN
                DISPLAY "Account not found"

            ELSE
                IF choice = 1 THEN
                    INPUT amount

                    IF amount > 0 THEN
                        balance = balance + amount
```

```
                        DISPLAY "Deposit successful"
                    ELSE
                        DISPLAY "Invalid amount"
                    END IF

                ELSE IF choice = 2 THEN
                    INPUT amount

                    IF amount <= 0 THEN
                        DISPLAY "Invalid amount"
                    ELSE IF amount > balance THEN
                        DISPLAY "Insufficient balance"
                    ELSE
                        balance = balance - amount
                        DISPLAY "Withdrawal successful"
                    END IF

                ELSE IF choice = 3 THEN
                    DISPLAY accountNumber
                    DISPLAY name
                    DISPLAY balance
                END IF
            END IF

        ELSE IF choice != 4 THEN
            DISPLAY "Invalid choice"
        END IF

    UNTIL choice = 4

    DISPLAY "Thank you for using our banking system"

END
```

3. Sample solution

Main.java: receives input and displays the menu.

```java
package Practice_5;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of customers: ");
        int n = sc.nextInt();
        sc.nextLine();

        if (n <= 0) {
            System.out.println("Number must be positive!");
            sc.close();
            return;
        }
        Bank bank = new Bank(n);

        for (int i = 0; i < n; i++) {
            System.out.println("\nCustomer " + (i + 1));
            String accountNumber;
            while (true) {
                System.out.print("Enter 14-digit account number: ");
                accountNumber = sc.nextLine();

                if (accountNumber.matches( regex: "\\d{14}")) {
                    break;
                }

```

```
                System.out.println("Account number must contain 14 digits!");
            }
            System.out.print("Enter name: ");
            String name = sc.nextLine();

            System.out.print("Enter initial balance: ");
            double balance = sc.nextDouble();
            sc.nextLine();

            while (balance < 0) {
                System.out.print("Balance cannot be negative. Enter again: ");
                balance = sc.nextDouble();
                sc.nextLine();
            }
            Customer customer = new Customer(
                    accountNumber, name, balance
            );

            bank.addCustomer(customer);
        }
        int choice;
        do {
            System.out.println("\n===== BANKING SYSTEM =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Display account details");
            System.out.println("4. Exit");
            System.out.print("Choose an option: ");

            choice = sc.nextInt();
            sc.nextLine();

            if (choice >= 1 && choice <= 3) {
                System.out.print("Enter account number: ");
                String accountNumber = sc.nextLine();

                Customer customer = bank.findCustomer(accountNumber);

                if (customer == null) {
                    System.out.println("Account not found!");
                    continue;
                }

                switch (choice) {
                    case 1:
                        System.out.print("Deposit amount: ");
                        customer.deposit(sc.nextDouble());
                        sc.nextLine();
                        break;

                    case 2:
                        System.out.print("Withdrawal amount: ");
                        customer.withdraw(sc.nextDouble());
                        sc.nextLine();
                        break;

                    case 3:
                        customer.displayDetails();
                        break;
                }
            } else if (choice != 4) {
                System.out.println("Invalid choice!");
            }

        } while (choice != 4);

        System.out.println("Thank you for using our banking system!");
        sc.close();
    }
}
```

- **Bank bank = new Bank(n):** creates a Bank object to store customer records.
- **for (int i = 0; i < n; i++):** loops to input each customer's account number, name, and initial balance.
- **Customer customer = new Customer(accountNumber, name, balance):** creates a Customer object with the customer's information.
- **bank.addCustomer(customer):** adds the customer object to the bank's array.
- do-while loop: repeatedly displays the banking menu until the user chooses to exit.
- **bank.findCustomer(accountNumber):** searches for a customer using their account number.
- **customer.deposit(amount):** adds money to the customer's balance after validating the amount.
- **customer.withdraw(amount):** subtracts money from the balance if the amount is valid and sufficient funds are available.
- **customer.displayDetails():** displays the customer's account number, name, and balance.
- **switch (choice):** executes the selected banking operation based on the user's menu choice.

Customer.java: stores customer information and performs deposits and withdrawals.

```java
package Practice_5;

public class Customer {  7 usages
    private String accountNumber;  3 usages
    private String name;  2 usages
    private double balance;  5 usages

    public Customer(String accountNumber, String name, double balance) {  1 usage
        this.accountNumber = accountNumber;
        this.name = name;
        this.balance = balance;
    }

    public String getAccountNumber() {  1 usage
        return accountNumber;
    }

    public void deposit(double amount) {  1 usage
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposit successful!");
        } else {
            System.out.println("Invalid amount!");
        }
    }

    public void withdraw(double amount) {  1 usage
        if (amount <= 0) {
            System.out.println("Invalid amount!");
        } else if (amount > balance) {
            System.out.println("Insufficient balance!");
        } else {
            balance -= amount;
            System.out.println("Withdrawal successful!");
        }
    }

    public void displayDetails() {  1 usage
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Name: " + name);
        System.out.printf("Balance: $%.2f%n", balance);
    }
}
```

- **public Customer(String accountNumber, String name, double balance):** constructor that initializes the customer's account number, name, and balance.
- **public String getAccountNumber():** returns the customer's account number for searching.
- **public void deposit(double amount):** checks whether the deposit amount is positive and adds it to the balance.
- **public void withdraw(double amount):** checks whether the withdrawal amount is valid and whether the customer has enough money before deducting it.

Bank.java: stores customer objects in an array and searches for accounts.

```java
package Practice_5;

public class Bank {  2 usages
    private Customer[] customers;  5 usages
    private int count;  5 usages

    public Bank(int size) {  1 usage
        customers = new Customer[size];
        count = 0;
    }

    public void addCustomer(Customer customer) {  1 usage
        if (count < customers.length) {
            customers[count] = customer;
            count++;
        } else {
            System.out.println("Customer list is full!");
        }
    }

    public Customer findCustomer(String accountNumber) {  1 usage
        for (int i = 0; i < count; i++) {
            if (customers[i].getAccountNumber().equals(accountNumber)) {
                return customers[i];
            }
        }
        return null;
    }
}
```

- **private Customer[] customers:** stores an array of Customer objects.
- **private int count:** tracks the number of customers currently stored in the array.
- **public Bank(int size):** constructor that initializes the customer array with the specified capacity and sets count to zero.
- **public void addCustomer(Customer customer):** adds a customer object to the array if space is available; otherwise, displays a message that the list is full.
- **public Customer findCustomer(String accountNumber):** uses a for loop to search for a customer by account number. Returns the matching customer if found, or null if the account does not exist.

Output:

```
Enter number of customers: 2

Customer 1
Enter 14-digit account number: 123456789012
Account number must contain 14 digits!
Enter 14-digit account number: 12345678901234
Enter name: Robert
Enter initial balance: 54

Customer 2
Enter 14-digit account number: 32476834672872
Enter name: Heng
Enter initial balance: -5
Balance cannot be negative. Enter again: 78

===== BANKING SYSTEM =====
1. Deposit
2. Withdraw
3. Display account details
4. Exit
Choose an option: 5
Invalid choice!

===== BANKING SYSTEM =====
1. Deposit
2. Withdraw
3. Display account details
4. Exit
Choose an option: 1
Enter account number: 12345678901234
Deposit amount: 15
Deposit successful!

===== BANKING SYSTEM =====
1. Deposit
2. Withdraw
3. Display account details
4. Exit
Choose an option: 4
Thank you for using our banking system!
```

Example1

```
Enter number of customers: 3

Customer 1
Enter 14-digit account number: 12345678901234
Enter name: Teang
Enter initial balance: -98
Balance cannot be negative. Enter again: 54

Customer 2
Enter 14-digit account number: 32742
Account number must contain 14 digits!
Enter 14-digit account number: 23145678901234
Enter name: Vuth
Enter initial balance: 67

Customer 3
Enter 14-digit account number: 09876543210987
Enter name: roth
Enter initial balance: 98

===== BANKING SYSTEM =====
1. Deposit
2. Withdraw
3. Display account details
4. Exit
Choose an option: 3
Enter account number: 12345678901234
Account Number: 12345678901234
Name: Teang
Balance: $54.00

===== BANKING SYSTEM =====
1. Deposit
2. Withdraw
3. Display account details
4. Exit
Choose an option: 2
Enter account number: 09876543210987
Withdrawal amount: 100
Insufficient balance!
```

Example2

THE END